STAT 405 Final Project

Alec Neme (84761030), Mikayla Liang (87701603)

2026-03-15

Abstract

Investor confidence is fundamentally tied to market stability, as sudden transitions into high-volatility regimes expose investors to significant risk. This study explores the use of Bayesian inference methods in detecting and predicting these volatile periods. Using historical data from Yahoo Finance spanning January 1998 to the present, the comparative models identify latent shifts in market behaviour driven by rapid economic change. [Insert results].

Introduction

Stock market volatility poses significant challenges for investors, particularly during periods of economic instability. Accurately detecting these periods is critical as it allows investors to anticipate risk and make confident investment decisions. Historically, forecasting has relied on frequentist methodologies, such as ARIMA and GARCH. However, these models are limited by their inability to incorporate prior context or macroeconomic indicators. In contrast, Bayesian approaches integrate prior probability distributions and typically achieve greater accuracy in predicting volatility.

To determine how to best capture latent shifts in the market, this study compares two Bayesian methods: a standard Hidden Markov Model (HMM) and a Hierarchical Hidden Markov Model (HHMM). The HMM models the market as switching between two latent states to capture sudden shifts in market behaviour. The HHMM extends this by introducing a hierarchical structure to the emission variances. Instead of relying on rigid, fixed priors that are highly sensitive to the scale of log returns, the HHMM utilizes a data-driven hyperparameter to dictate the rate of volatility. This flexible structure allows the model to better accommodate the "fat tails" inherent in financial data, preventing single-week outliers from hijacking the latent state estimation and producing much more accurate, sustained regime probabilities. Both models are fitted using two inferential methods: Hamiltonian Monte Carlo (HMC), which produces an accurate posterior by sampling, and Variational Inference (VI), which approximates the posterior through optimization. By applying both methods to each model, we can evaluate the tradeoff between accuracy and computational efficiency.

The analysis spans three major stock market crises: the Dot-com Bubble, the 2008 Financial Crisis, and the 2020 COVID-19 crash. The S&P 500 Index (ˆGSPC) serves as the primary dataset for this study, as it is widely regarded as the most complete benchmark of U.S. equity market performance. Given that these crises were predominantly driven by U.S. market dynamics, the S&P 500 accurately reflects the desired market volatility.

Literature Review

[Description of sections]

Methodology

To validate the accuracy of our approach, we may utilize the CBOE Volatility Index (VIX), allowing us to compare our model's estimated latent volatility against Wall Street's implied volatility. Additionally, assets such as sector-specific ETFs (XLK, XLF) and a safe-haven treasury bond (TLT) will be incorporated to test if the model's regime-switching logic generalizes reliably across different market segments, as well as sector-specific insight of each crisis. To reduce the computational burden while preserving essential volatility clustering, the daily data will be mutated to only include the final trading day of each week to establish consistent weekly boundaries. Finally, these weekly prices will be transformed into weekly log returns, calculated as $Y_t = \ln(P_t/P_{t-1})$. This transformation is critical, as it ensures a stationary distribution of the time series which facilitates its modelling.

Definition of Indexes

- ^GSPC (S&P 500 Index): Top 500 stocks in the U.S. This is the observation that we will most likely use.
- ^VIX (CBOE Volatility Index): Market volatility measure that we can use for model validation.
- XLK (Technology Select Sector SPDR Fund): An ETF containing all the tech stocks in the S&P 500. Maybe use for dot com crash?
- XLF (Financial Select Sector SPDR Fund): An ETF containing all the financial stocks in the S&P 500 (banks, insurance, etc.). Check for 2008 Financial Crisis?
- TLT (iShares 20+ Year Treasury Bond ETF): This tracks long-term US government bonds. "Safe haven" asset that normally behaves opposite to stocks. Does the model still work with this?

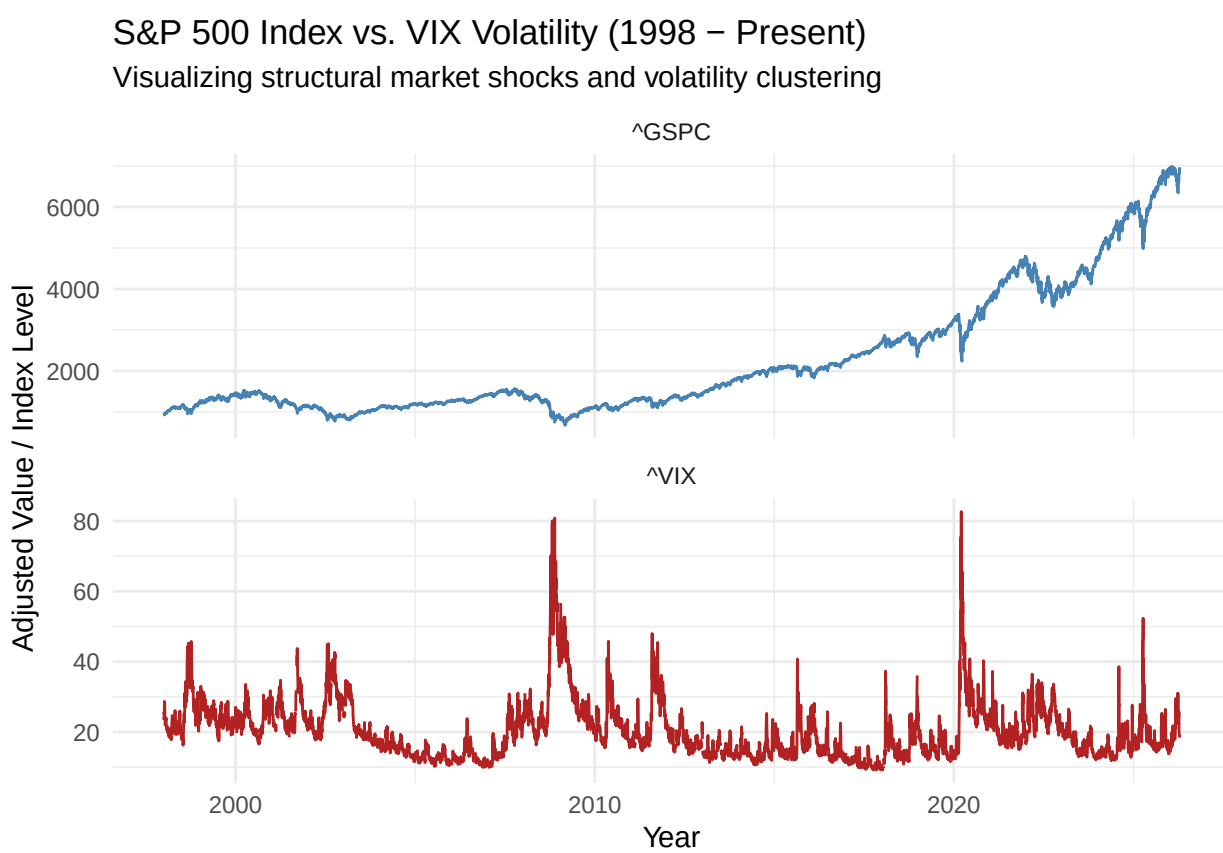
```r
project_tickers <- c("^GSPC", "^VIX", "XLK", "XLF", "TLT")
market_data <- tq_get(project_tickers,
                      get  = "stock.prices",
                      from = "1998-01-01")

weekly_returns <- market_data %>%
  mutate(week = floor_date(date, "week")) %>%
  group_by(symbol, week) %>%
  arrange(date) %>%
  slice_tail(n = 1) %>%
  ungroup()


weekly_log_returns <- weekly_returns %>%
  group_by(symbol) %>%
  arrange(date) %>%
  mutate(
    previous_price = lag(adjusted),
    log_return = log(adjusted / previous_price)
  ) %>%
  filter(!is.na(log_return)) %>%
  select(symbol, date, adjusted, log_return)

head(weekly_log_returns)
```

```
## # A tibble: 6 x 4
```

```
## # Groups:   symbol [2]
##   symbol date       adjusted log_return
##   <chr>  <date>        <dbl>      <dbl>
## 1 ^GSPC  1998-01-09    928.    -0.0498
## 2 ^VIX   1998-01-09     28.7    0.203
## 3 ^GSPC  1998-01-16    962.     0.0358
## 4 ^VIX   1998-01-16     21.6   -0.282
## 5 ^GSPC  1998-01-23    958.    -0.00409
## 6 ^VIX   1998-01-23     23.3    0.0743
```

EDA

```r
market_data %>%
  filter(symbol %in% c("^GSPC", "^VIX")) %>%
  ggplot(aes(x = date, y = adjusted, color = symbol)) +
  geom_line(show.legend = FALSE) +
  facet_wrap(~symbol, scales = "free_y", ncol = 1) +
  theme_minimal() +
  labs(title = "S&P 500 Index vs. VIX Volatility (1998 - Present)",
       subtitle = "Visualizing structural market shocks and volatility clustering",
       x = "Year",
       y = "Adjusted Value / Index Level") +
  scale_color_manual(values = c("^GSPC" = "steelblue", "^VIX" = "firebrick"))
```



The visualization highlights a clear inverse correlation: significant price contractions in the S&P 500

(^{G}SPC) correspond directly to sharp spikes in the VIX. These synchronized movements clearly mark the three major structural shocks identified in our proposal. The abrupt, vertical nature of these volatility surges visually mimics discrete state transitions, reinforcing our decision to utilize a Hidden Markov Model (HMM).

Furthermore, the raw S&P 500 series exhibits a distinct upward trend over the 25-year period, indicating it is non-stationary. By transforming these prices into weekly log returns, $Y_t = \ln(P_t/P_{t-1})$, we remove this drift and stabilize the variance. This transformation is mathematically critical; it ensures our HMM operates on a stationary distribution, which is a necessary condition for the model to reliably infer the "Calm" and "Crisis" latent states over the long term.

Hidden Markov Model

The Likelihood

Let Y_t represent the weekly log return of the index at time t, and let $X_t \in \{1, 2\}$. State 1 represents a "Calm/Normal" market regime, and State 2 represents a "Crisis" regime.

$$Y_t \mid X_t = i \sim \mathcal{N}(0, \sigma_i^2)$$

Where:

$$\sigma_1 \sim Exponential(1)$$
$$\sigma_{diff} \sim Exponential(1)$$
$$\sigma_2 = \sigma_1 + \sigma_{diff}$$

The Latent State Dynamics (The Regime Switching)

The sequence of unobserved market states follows a first-order Markov process. The probability distribution of tomorrow's state, X_t, is explicitly conditioned on today's state, X_{t-1}.

Latent States

$$X_t \mid X_{t-1} = 1 \sim \text{Categorical}(P_{11}, 1 - P_{11})$$
$$X_t \mid X_{t-1} = 2 \sim \text{Categorical}(1 - P_{22}, P_{22})$$

Transition Probabilities

$$P = \begin{pmatrix} P_{11} & 1 - P_{11} \\ 1 - P_{22} & P_{22} \end{pmatrix}$$

where,

$$P_{11} \sim \text{Beta}(50, 2)$$
$$P_{22} \sim \text{Beta}(20, 2)$$

We chose these priors for the diagonal probabilities, since we expect the markets to persist, and we want the model to ignore sudden weekly fluctuations in volatility, and focus on the bigger picture of being in a certain latent state. The probability of being stuck in a crisis state is modelled to be less sticky than when in a calm state, since people and policy are actively trying to avoid being in a crisis.

The Marginalized Joint Likelihood

Because Stan's Hamiltonian Monte Carlo (HMC) algorithm relies on continuous gradient calculations, it cannot directly sample discrete latent parameters. Consequently, we must apply Rao-Blackwellization by computing the marginal posterior, effectively summing out the discrete state sequence $X_{1:T}$.

Over the whole dataset, the brute-force marginalized joint likelihood requires summing over the two states $\{1, 2\}$ for all T weeks:

$$p(Y_{1:T}|\theta, P) = \sum_{x_1=1}^{2} \sum_{x_2=1}^{2} \cdots \sum_{x_T=1}^{2} p(Y_{1:T}, X_{1:T} = x_{1:T}|\theta, P)$$

By factoring the joint distribution into the initial state probabilities π, the transition matrix P, and the state-dependent emission likelihoods, we obtain the fully expanded equation:

$$p(Y_{1:T}|\theta, P) = \sum_{x_1=1}^{2} \cdots \sum_{x_T=1}^{2} \left[\pi_{x_1} p(Y_1|\mu_{x_1}, \sigma^2_{x_1}) \prod_{t=2}^{T} P_{x_{t-1},x_t} p(Y_t|\mu_{x_t}, \sigma^2_{x_t}) \right]$$

This exhaustive summation involves 2^T unique possible paths. Computing this $\mathcal{O}(2^T)$ sum at every MCMC step is computationally expensive.

Rao-Blackwellization via the Forward Algorithm

To resolve this computational bottleneck, we utilize the Forward Algorithm. The Forward Algorithm is a dynamic programming approach that relies on the conditional independence of the HMM. Instead of calculating the probability of every single distinct path from week 1 to week T independently, the algorithm calculates the probability sequentially and caches the intermediate results.

Because the Markov property dictates that the future state depends only on the present state, we do not need to track how the market arrived at a certain state, only the accumulated probability mass of being there. We define a forward variable, $\alpha_t(j)$, representing the joint probability of observing the sequence of returns up to time t and currently being in state j:

$$\alpha_t(j) = p(Y_{1:t}, X_t = j|\theta, P)$$

The algorithm computes this dynamically in three steps:

1. Base Case ($t = 1$):

$$\alpha_1(j) = \pi_j p(Y_1|\mu_j, \sigma^2_j) \quad \text{for } j \in \{1, 2\}$$

2. Recursive Step ($t = 2, \ldots, T$): By summing the cached probabilities from the previous week ($\alpha_{t-1}(i)$) and multiplying by the transition probability P_{ij}, we find the total probability of arriving at state j. We then multiply by the new emission likelihood:

$$\alpha_t(j) = p(Y_t|\mu_j, \sigma^2_j) \sum_{i=1}^{2} \alpha_{t-1}(i) P_{ij}$$

3. Termination (Marginalized Likelihood): At the final week T, the algorithm has compressed the 2^T paths into just two values: $\alpha_T(1)$ and $\alpha_T(2)$, representing the joint probability of the entire dataset ending in State 1 and State 2, respectively. By the Law of Total Probability, since the market must end in one of these two mutually exclusive states, the final marginalized likelihood is simply their sum:

$$L(\theta, P; Y_{1:T}) = p(Y_{1:T}|\theta, P) = \sum_{j=1}^{2} \alpha_T(j)$$

This reduces the time complexity from $\mathcal{O}(2^T)$ to $\mathcal{O}(T)$.

Log-Space Implementation

While the Forward Algorithm solves the theoretical complexity, implementing it directly may cause underflow. Multiplying hundreds of small probability values quickly results in a number smaller than a computer's floating-point limit (e.g., 10^{-308}), causing the software to round the total likelihood to zero.

To maintain numerical stability, Stan evaluates the marginalized posterior strictly in log-space. We define the log-forward variable as $\log \alpha_t(j)$.The base case becomes a simple addition:

$$\log \alpha_1(j) = \log \pi_j + \log p(Y_1|\mu_j, \sigma_j^2)$$

The recursive step requires taking the log of a sum, which Stan handles using the stable log_sum_exp function:

$$\log \alpha_t(j) = \log p(Y_t|\mu_j, \sigma_j^2) + \log \left(\sum_{i=1}^{2} \exp(\log \alpha_{t-1}(i) + \log P_{ij}) \right)$$

Finally, the total marginalized log-likelihood is the log-sum-exp of the final terminal states:

$$\log L(\theta, P; Y_{1:T}) = \log \left(\sum_{j=1}^{2} \exp(\log \alpha_T(j)) \right)$$

Implmentation of HMM

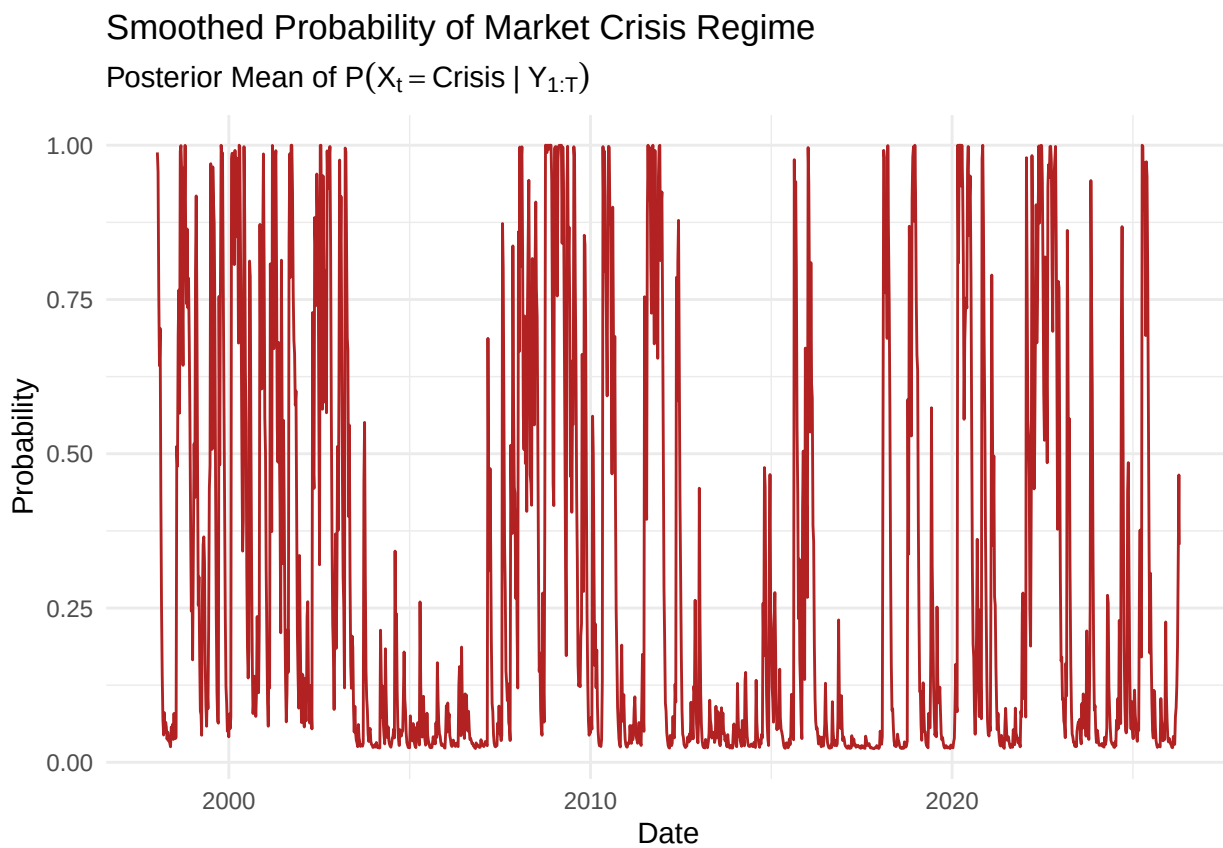
```r
model <- cmdstan_model("hmm_volatility.stan")

gspc_returns <- weekly_log_returns %>%
  filter(symbol == "^GSPC") %>%
  arrange(date)

stan_data <- list(
  T = nrow(gspc_returns),
  Y = gspc_returns$log_return
)

fit <- model$sample(
  data = stan_data,
  chains = 4,
  parallel_chains = 4,
```

```r
  iter_warmup = 1000,
  iter_sampling = 1000,
  adapt_delta = 0.95,
  seed = 405,
  refresh = 0
)
```

```
## Running MCMC with 4 parallel chains...
```

```
## Chain 4 finished in 78.9 seconds.
## Chain 2 finished in 101.3 seconds.
## Chain 1 finished in 107.3 seconds.
## Chain 3 finished in 108.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 99.1 seconds.
## Total execution time: 109.0 seconds.
```

```r
prob_crisis_draws <- fit$draws("prob_crisis", format = "matrix")

mean_prob_crisis <- colMeans(prob_crisis_draws)

data.frame(
  date = gspc_returns$date,
  prob = mean_prob_crisis
) %>%
  ggplot(aes(x = date, y = prob)) +
  geom_line(color = "firebrick") + # A distinct color for the smoothed probabilities
  theme_minimal() +
  labs(
    title = "Smoothed Probability of Market Crisis Regime",
    subtitle = expression("Posterior Mean of " * P(X[t] == Crisis ~ "|" ~ Y["1:T"])),
    x = "Date",
    y = "Probability"
  )
```




```r
fit$summary(c("P11", "P22", "sigma_1", "sigma_2"))
```

```
## # A tibble: 4 x 10
##   variable   mean median       sd      mad     q5    q95  rhat ess_bulk ess_tail
##   <chr>     <dbl>  <dbl>    <dbl>    <dbl>  <dbl>  <dbl> <dbl>    <dbl>    <dbl>
## 1 P11      0.965  0.965  0.00873  0.00855  0.949  0.978   1.00    1979.    2388.
## 2 P22      0.916  0.918  0.0211   0.0210   0.879  0.947   1.00    1708.    2447.
## 3 sigma_1  0.0151 0.0151 0.000548 0.000538 0.0143 0.0160  1.00    1685.    2138.
## 4 sigma_2  0.0387 0.0386 0.00184  0.00183  0.0359 0.0420  1.00    2047.    1869.
```

Model 1 (Naive HMM) Results & Discussion

Starting Out (Forward Pass Only): * We started by just using the forward algorithm to calculate "filtered" probabilities. * Because this method only looks at past and present data, it was way too reactive. The model basically panicked at every little market dip, giving us super jittery results instead of identifying actual, long-term crisis periods.

Bringing in the Backward Pass: * To smooth out the jitter, we added the backward algorithm to calculate "smoothed" probabilities. * Look at Rabiner (1989) and 2020 MDPI paper * The goal here was to let the model look at the *entire* timeline of data at once. With the benefit of hindsight, we hoped it would realize that massive crashes (like 2008) are sustained events, not just random one-week hiccups.

- **Still Jittery (The "Barcode" Effect):** * Even with the backward pass and telling the model to strongly prefer staying in whatever state it was currently in (adding "stickiness"), the plot still looked like a crazy barcode. It kept rapidly flipping back and forth between 0 and 1 almost every week.

- **The Diagnosis (Prior Sensitivity):**
- We realized the issue came down to our rigid setup and how we scaled the data. Our fixed priors expected normal-sized numbers, but our raw log returns were tiny decimals with extreme, unpredictable outliers.
- When the market had a wild week, the normal math basically broke down. To survive the calculation, the model was forced to jump into the "crisis" state for just that single week, completely ignoring our rules about being sticky. This proved that our naive baseline model was way too sensitive and just couldn't handle the true market volatility.

Hierarchical Hidden Markov Model (Model 2)

While our baseline HMM successfully captures discrete regime shifts, relying on fixed hyper-parameters (such as $\sigma_i \sim \text{Exponential}(1)$) introduces a vulnerability: the posterior may be overly sensitive to these rigid prior assumptions. To mitigate this and capture the true scale of market volatility driven by the data, we introduce a Hierarchical Hidden Markov Model (HHMM).

Because weekly log returns are practically indistinguishable from zero and a "crisis" is defined primarily by an explosion in variance rather than a shift in the mean, we simplify the emission model by assuming a constant global mean (μ) across both states. This isolates the regime-switching mechanics entirely on the market volatility.

Hierarchical Parameter Definitions

The observed weekly log returns are modeled with a constant global mean and state-dependent hierarchical variances:

$$Y_t \mid X_t = i \sim \mathcal{N}(\mu, \sigma_i^2)$$

Rather than fixing the rate of the Exponential prior, we introduce a hyperparameter λ drawn from a Gamma distribution:

$$\lambda \sim \text{Gamma}(2, 0.1)$$

$$\sigma_1 \sim \text{Exponential}(\lambda)$$

$$\sigma_2 \sim \text{Exponential}(\lambda) \quad \text{subject to } \sigma_2 > \sigma_1$$

Justification of Priors and Constraints

The selection of the prior $\lambda \sim \text{Gamma}(2, 0.1)$ is because of the following: * **Shape Control ($\alpha = 2$):** An Exponential distribution has its mode strictly at zero. By using a Gamma shape parameter of 2, we introduce a gentle peak that mathematically pushes the sampler away from exactly zero. * **Weakly Informative Rate ($\beta = 0.1$):** Because the rate parameter acts as an inverse scale, setting it to 0.1 spreads the probability mass out over a wide, flat tail. This provides a weakly informative prior that allows the true, extreme volatility of events like the 2008 Financial Crisis and the COVID-19 crash to be driven by the data rather than prior bias.

Finally, the explicit ordering constraint ($\sigma_2 > \sigma_1$) is mathematically required to resolve the "label switching", same as in our naive model.

Implmentation of HHMM

```r
model_hierarchical <- cmdstan_model("hhmm_volatility.stan")
```

```
## Warning in readLines(stan_file): incomplete final line found on
## 'hhmm_volatility.stan'
```

```r
stan_data_hierarchical <- list(
  T = nrow(gspc_returns),
  Y = gspc_returns$log_return
)

fit_hierarchical <- model_hierarchical$sample(
  data = stan_data_hierarchical,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  adapt_delta = 0.95, # High adapt_delta helps with hierarchical model geometry
  seed = 405
)
```

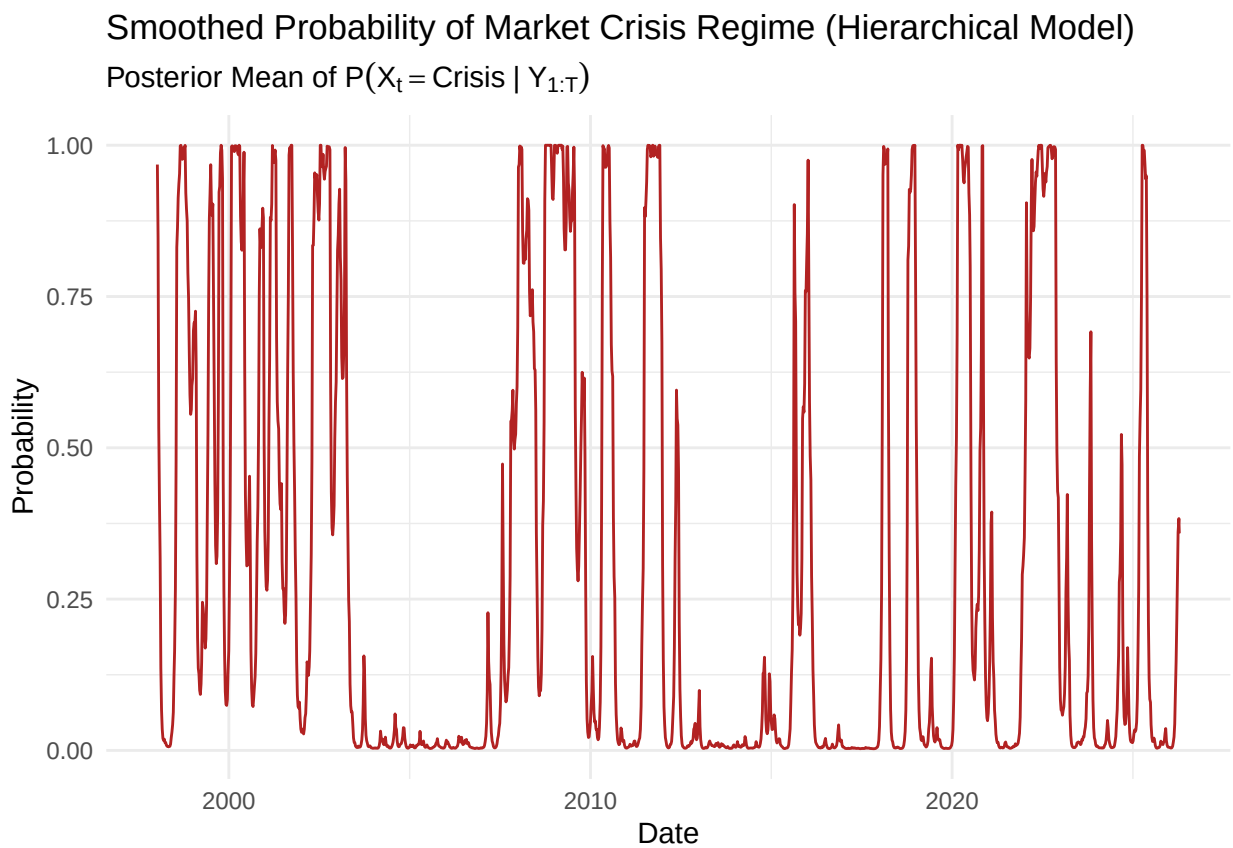
```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/alnem/AppDa

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi

## Chain 1

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/alnem/AppDa

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi

## Chain 1

## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
```

```
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/alnem/AppDa

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi

## Chain 2

## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/alnem/AppDa

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or mi

## Chain 3

## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/alnem/AppDa

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi

## Chain 4

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/alnem/AppDa

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi

## Chain 4

## Chain 4 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 1 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 1 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 3 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 4 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 1 Iteration:  300 / 2000 [ 15%]  (Warmup)
```

```
## Chain 3 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 2 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 1 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 1 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 4 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 2 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 3 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 1 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 4 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 1 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 1 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 3 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 3 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 1 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 2 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 3 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 4 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 3 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 4 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 2 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 2 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 4 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 1 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 2 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 4 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1400 / 2000 [ 70%]  (Sampling)
```

```
## Chain 4 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 78.3 seconds.
## Chain 2 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 4 finished in 80.1 seconds.
## Chain 2 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 3 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 2 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 91.0 seconds.
## Chain 2 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 2 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 103.5 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 88.3 seconds.
## Total execution time: 103.6 seconds.
```

```r
prob_crisis_draws <- fit_hierarchical$draws("prob_crisis", format = "matrix")

mean_prob_crisis <- colMeans(prob_crisis_draws)

data.frame(
  date = gspc_returns$date,
  prob = mean_prob_crisis
) %>%
  ggplot(aes(x = date, y = prob)) +
  geom_line(color = "firebrick") +
  theme_minimal() +
  labs(
    title = "Smoothed Probability of Market Crisis Regime (Hierarchical Model)",
    subtitle = expression("Posterior Mean of " * P(X[t] == Crisis ~ "|" ~ Y["1:T"])),
    x = "Date",
    y = "Probability"
  )
```



References

https://vlab.stern.nyu.edu/docs/volatility/GARCH https://www.ijirmf.com/wp-content/uploads/IJIRMF202410007-min.pdf https://www.jstor.org/stable/1392199?if_data=e30%3D&seq=1

Rabiner (1989): https://www.cs.ubc.ca/~murphyk/Bayes/rabiner.pdf

MDPI With similar approach: https://www.mdpi.com/1911-8074/13/12/311